

Statistical Analysis Using Python

Dr Jerin Paul
Assistant Professor
Department of Statistics
Vimala College (Autonomous), Thrissur

July 2026

Contents

Preface	5
<i>A Textbook for FYUGP Programme Vth Sem BSc Statistics, Vimala College (Autonomous), Thrissur</i>	5
Acknowledgements	6
Course Information	6
1 Introduction to Python Programming	9
1.1 The Interactive Python Environment	9
1.2 Basic Syntax and Program Readability	10
1.3 Variables and Dynamic Typing	12
1.4 Arithmetic Operators and Expressions	12
1.5 Reading User Input and Type Conversion	13
1.6 Working with Strings and Formatting	14
1.7 Control Flow: Decision Making	16
1.8 Control Flow: Loops	18
1.9 Interpreting Error Messages	20
1.10 Importing Modules	22
1.11 Function Fundamentals	23
Practice Exercises	26
2 Data Manipulation with Pandas	27
2.1 Introduction to Pandas	27
2.2 The Pandas Series	28

2.3	The Pandas DataFrame	30
2.4	Data Operations	31
2.5	Descriptive Statistics	33
	Practice Exercises	35
3	Data Visualization	37
3.1	Basic Charts with Matplotlib	38
3.2	Preparing the College Data for Plotting	41
3.3	Statistical Charts with Seaborn	44
3.4	Advanced Plots	47
	Practice Exercises	51
4	Statistical Data Analysis	53
4.1	Generating Simulated Data	53
4.2	Hypothesis Testing: The Courtroom Analogy	56
4.3	Checking for Normality: The Gatekeeper	56
4.4	One-Sample Tests	57
4.5	Two-Sample Tests: Comparing Two Separate Groups	58
4.6	Paired Tests: Comparing the Same People Twice	60
4.7	Comparing Three or More Groups: ANOVA	61
4.8	Correlation	64
4.9	Simple Linear Regression	65
4.10	Multiple Linear Regression	67
4.11	Logistic Regression	67
	Practice Exercises	69
	Glossary	71
	References	75

Preface

A Textbook for FYUGP Programme Vth Sem BSc Statistics, Vimala College (Autonomous), Thrissur

This textbook has been written to meet the requirements of the **Four Year Undergraduate Programme (FYUGP)** of Calicut University for the Skill Enhancement Course on **Statistical Analysis Using Python**.

The book is organised into four chapters aligned with the first four modules of the syllabus:

Chapter	Title
1	Introduction to Python Programming
2	Data Manipulation with Pandas
3	Data Visualization
4	Statistical Data Analysis

Each chapter opens with learning objectives. Key terms are **bolded** when first introduced. Every topic is explained in plain language with fully worked code examples, so that a student can prepare for the written examination directly from this book. Selected practice exercises are provided at the end of each chapter for students who wish to try the code themselves.

All code in this book is written for **Python 3** and is intended to be run in **Google Colab**, a free cloud-based notebook environment. No software installation is required: a student needs only a web browser and a Google account.

The code examples are shown as static listings together with their expected output. Students are encouraged to type each example into a Colab cell and run it to confirm the result.

Acknowledgements

The author acknowledges the reference material of the course, in particular the works of Wes McKinney, Jake VanderPlas, Charles Severance, Thomas Haslwanter, and Ossama Embarak, from which the topics of this course are drawn.

Course Information

Course Details

Programme	B.Sc. Statistics
Course Code	STA5FS112
Course Title	Statistical Analysis Using Python
Type of Course	SEC
Semester	V
Academic Level	100–199

Credit Structure

Credit	Lecture/Week	Tutorial/Week	Practical/Week	Total Hours
3	3	—	—	45

Pre-requisites: HSE level Mathematics Course

Detailed Syllabus

Module I — Introduction to Python Programming (12 Hrs)

1. Interactive Python Environment: Jupyter notebooks, basic syntax, interactive shell.
2. Data Fundamentals: Variables, assignments, arithmetic operators, expressions.
3. Program Readability: Comments in code, interpreting error messages.
4. Modular Programming: Importing modules, control flow statements.
5. Function Fundamentals: Built-in and user-defined functions, arguments, return values, formal vs. actual parameters, named arguments.

Module II — Data Manipulation with Pandas (10 Hrs)

6. Pandas Introduction: Data Series, Data Frames.
7. Data Operations: Importing, manipulating, merging, analyzing, and exporting Data Frames.
8. Descriptive Statistics: Exploratory data analysis techniques.

Module III — Data Visualization (8 Hrs)

9. Data Visualization Libraries: Matplotlib, Seaborn, Plotly, ggplot, Geoplotlib, Pandas.
10. Plot-I: Line plot, bar plot, pie chart, box plot, histogram, strip plot, swarm plot.
11. Plot-II: Scatter plot, heatmap, density plot, cumulative frequencies, error bars.

Module IV — Statistical Data Analysis Using statsmodels (18 Hrs)

12. Random number generation and correlation.
13. Hypothesis testing: one-sample, two-sample, and paired t-tests; one-way and two-way ANOVA; non-parametric tests.
14. Modelling: simple and multiple linear regression, and logistic regression.

The Dataset Used in This Book

Chapters 2 and 3 use a single dataset, **College_Data.csv**, so that all examples are consistent and self-contained. The data were collected from postgraduate students of three different colleges in Kerala. The variables are described below.

Variable	Description
Gender	0 – Male, 1 – Female
Locality	0 – Urban, 1 – Rural
Income	0 – Less than 10000, 1 – 10000 to 20000, 2 – 20000 to 40000, 3 – Above 40000
College	1 – College A, 2 – College B, 3 – College C
SSLC	Percentage of marks in SSLC
PlusTwo	Percentage of marks in Plus Two
Degree	Percentage of marks in Degree

To use this dataset in Google Colab, upload the file **College_Data.csv** to the notebook session before running the examples in Chapters 2 and 3.

Chapter 1

Introduction to Python Programming

Learning Objectives

After studying this chapter, a student will be able to:

- describe the interactive Python environment and the structure of a Jupyter/Colab notebook;
- write and execute basic Python statements, comments, and correctly indented code;
- declare variables and explain dynamic typing, along with the rules for naming variables;
- use arithmetic operators and evaluate expressions;
- read input from the user and convert it between data types;
- work with strings and format output;
- interpret common error messages and handle exceptions;
- import and use modules; and
- write control-flow statements, loops, and user-defined functions.

1.1 The Interactive Python Environment

Python is a high-level, general-purpose programming language. It is widely used because it is free, open-source, and runs on multiple platforms such as Windows, Mac, and Linux.

Python is an **interpreted language**, which means that an interpreter reads the source code and executes it directly, one instruction at a time. Because of this, Python can be used interactively: a student can type an instruction, press a key, and immediately see the result.

1.1.1 The Interactive Shell

The simplest way to interact with Python is through the **interactive shell**. When the shell is opened, it displays the prompt `>>>`, which is the interpreter’s way of asking, “What do you want me to do next?” A student can type an expression such as `3 + 4`, and Python will immediately evaluate it and display the result `7` on the next line before showing a new prompt.

1.1.2 Jupyter Notebooks and Google Colab

While the basic shell is useful, data analysis is usually carried out in a **Jupyter Notebook**. A Jupyter Notebook is an interactive, browser-based environment that allows a user to combine executable Python code, formatted text, mathematical equations, and visualizations, all within a single document.

A notebook is made up of **cells**. A programmer types code into a cell and executes it by pressing **Shift + Enter**. The output of that particular block of code appears directly beneath the cell. This encourages an interactive “*execute-explore*” style of working, in which a small piece of code is written, run, and its result examined before moving on.

Google Colab is a free, cloud-hosted version of a Jupyter Notebook. Because it runs in the cloud, it requires no installation of Python or any package on the student’s computer; only a web browser and a Google account are needed. Throughout this book, every code example is intended to be typed into a Colab cell and run with **Shift + Enter**. Colab behaves exactly like a Jupyter Notebook, so the terms may be used interchangeably.

The very first program a student usually writes simply displays a line of text on the screen:

```
print('Hello World')
```

Hello World

The built-in `print()` function displays whatever is placed inside its parentheses.

1.2 Basic Syntax and Program Readability

A Python program is a sequence of **statements**. A statement is a unit of code that the interpreter can execute, such as assigning a value to a variable or printing a result. When a statement is typed in a cell and executed, the interpreter reads it, checks its syntax, translates it, and produces a result.

1.2.1 Keywords

Python has a small built-in vocabulary of **reserved words**, also called **key-words**. Words such as `if`, `def`, `for`, `print`, and `while` have specific meanings to Python and cannot be used as variable names.

1.2.2 Comments

Comments are notes written for human readers to explain the code. They are completely ignored by the Python interpreter.

- A **single-line comment** begins with the hash symbol (`#`).
- A **multi-line comment**, often called a **docstring**, is enclosed within triple quotation marks (`"""..."""`).

```
# This is a single-line comment
print("Welcome to Statistical Analysis")  # this comment
    ↪ explains the line

"""
This is a multi-line comment.
It can span several lines.
"""
```

1.2.3 Statement Termination

In most cases, the end of a physical line marks the end of a statement, so lines do not need to be ended with a semicolon. If a statement is too long to fit on one line, the **line-continuation character**, a backslash (`\`), may be placed at the end of the line. Alternatively, any expression enclosed within parentheses (`()`), brackets [`[]`], or braces [`{}`] may naturally span several lines without a backslash.

1.2.4 Indentation

Indentation is one of the most important features of Python. Unlike many languages that use braces `{}` to group blocks of code, Python rigidly uses line indentation — the blank space to the left of a line — to determine where a block of code begins and ends. Every statement within a single block must be indented by the same amount. Indentation is a strict requirement, not merely a matter of style; incorrect indentation produces an error.

1.3 Variables and Dynamic Typing

A **variable** is a name that refers to a stored value in the computer's memory. It is helpful to think of a variable as a *label* or a *pointer* attached to an object, rather than a box that holds data.

1.3.1 Dynamic Typing

Python uses **dynamic typing**, which means a programmer does not need to declare in advance what type of data a variable will hold. The interpreter determines the type automatically when the program runs. Because of this, the same variable can hold an integer at one moment and a string at another:

```
x = 100          # x now refers to an integer
x = "Hello"     # the same x now refers to a string
```

No error occurs; the variable simply points to the new object.

1.3.2 Rules for Naming Variables

Python is strict about how variables are named:

- A name must start with a letter (a–z, A–Z) or an underscore (_).
- A name cannot start with a number. For example, `1plus` is invalid, but `plus1` is valid.
- A name cannot contain special characters such as @, \$, %, or spaces.
- Names are **case-sensitive**, so `Age`, `age`, and `AGE` are three different variables.
- A **reserved keyword** (such as `if`, `def`, `for`, `print`, or `while`) cannot be used as a variable name.

1.4 Arithmetic Operators and Expressions

Python can act as a powerful calculator. An **expression** is a combination of values, variables, and operators that the interpreter evaluates to produce a new value. The main arithmetic operators are listed below.

Operator	Meaning	Example	Result
+	Addition	4 + 3	7
-	Subtraction	4 - 3	1

Operator	Meaning	Example	Result
*	Multiplication	4 * 3	12
/	Floating-point division	13 / 5	2.6
//	Floor division	13 // 5	2
%	Modulus (remainder)	13 % 5	3
**	Exponentiation (power)	2 ** 3	8

- **Floating-point division** (/) performs ordinary division and gives a decimal result.
- **Floor division** (//) divides and drops the fractional remainder.
- **Modulus** (%) returns only the remainder of a division.
- **Exponentiation** (**) raises a number to a power.

Worked Example 1.1 — The Python Calculator

A student has 47 apples and wants to divide them evenly among 5 friends. The following program finds how many whole apples each friend gets and how many are left over.

```
apples = 47
friends = 5
apples_per_friend = apples // friends
apples_left_over = apples % friends
print("Each friend gets:", apples_per_friend)
print("Apples left over:", apples_left_over)
```

```
Each friend gets: 9
Apples left over: 2
```

Floor division (//) gives the number of whole apples per friend, and modulus (%) gives the remainder.

1.5 Reading User Input and Type Conversion

1.5.1 The input() Function

The built-in `input()` function pauses the program and prompts the user to type something from the keyboard:

```
name = input("Enter your name: ")
```

An important point is that `input()` **always returns the typed data as a string** (text), even when the user types a number. If a user types 10, Python stores it as the text "10".

1.5.2 Type Conversion (Casting)

To perform arithmetic on user input, the string must first be converted into a number. This is called **type conversion** or **casting**, and it uses built-in functions:

- `int()` converts a value to a whole number;
- `float()` converts a value to a number with a decimal point;
- `str()` converts a number back into a string.

Worked Example 1.2 — The Input Trap

This example shows why type conversion matters. The same input is used first without conversion and then with conversion.

```
# Without conversion: the string is repeated
user_val = input("Enter a number: ")
print("Without conversion:", user_val * 5)

# With conversion: real multiplication takes place
numeric_val = float(user_val)
print("With conversion:", numeric_val * 5)
```

If the user enters 4, the output is:

```
Without conversion: 44444
With conversion: 20.0
```

Because `input()` returns text, `user_val * 5` repeats the text five times. Only after converting to a number does `* 5` perform multiplication.

1.6 Working with Strings and Formatting

A **string** is a sequence of characters. Python allows strings to be written using single quotes (`'...'`), double quotes (`"..."`), or triple quotes (`'''...'''` or `"""..."""`).

Different quote styles are useful in different situations. If the text itself contains an apostrophe, the string can be wrapped in double quotes to avoid confusion, for example `"I'm learning Python"`. Triple quotes are used for strings that span several lines and for docstrings.

1.6.1 String Operations

- **Concatenation** (+) joins two strings end to end: "Hello" + "World" gives "HelloWorld".
- **Repetition** (*) repeats a string a given number of times: "Hi" * 3 gives "HiHiHi".

1.6.2 Escape Sequences

An **escape sequence** uses the backslash (\) to insert a special character inside a string:

- \n inserts a new line;
- \t inserts a horizontal tab.

1.6.3 Formatting Output

The `.format()` method inserts variables neatly into a string. Placeholders `{0}`, `{1}`, and so on are replaced, in order, by the values given to `.format()`:

```
name = "Anu"
age = 20
print("Hello {0}, you are {1} years old".format(name, age))
```

Hello Anu, you are 20 years old

A second, more modern way to do the same thing is the **f-string** (formatted string literal). An f-string is written by placing the letter **f** immediately before the opening quote. Any variable placed inside curly braces `{}` within the string is then replaced by its value. This is often the most convenient way to combine text and variables, and it is the style used in most of the examples that follow.

```
name = "Anu"
age = 20
print(f"Hello {name}, you are {age} years old")
```

Hello Anu, you are 20 years old

An arithmetic expression may also be placed directly inside the braces of an f-string:

```
weight = 72
print(f"Pay an additional charge for {weight - 25} kg")
```

Pay an additional charge for 47 kg

Worked Example 1.3 — Formatting a Bill

This program asks for an item, its price, and the quantity, then prints a neat receipt.

```
item = input("Enter item name: ")
price = float(input("Enter price per unit: "))
qty = int(input("Enter quantity: "))
total = price * qty
print("You bought {0} units of {1} at ${2} each. Total:
↪ ${3}".format(qty, item, price, total))
```

If the user enters Apple, 2.5, and 3, the output is:

You bought 3 units of Apple at \$2.5 each. Total: \$7.5

Note the use of `float()` for the price and `int()` for the quantity, so that the multiplication is numerical.

1.7 Control Flow: Decision Making

By default, a program executes its statements one after another, from top to bottom. **Control-flow statements** allow a program to make decisions and execute particular blocks of code only when a condition is met.

An important rule applies to all of these statements: the condition line ends with a colon (:), and the block of code that belongs to it must be **indented**.

1.7.1 The if Statement

The `if` statement evaluates a condition. If the condition is `True`, the indented block below it is executed; if it is `False`, the block is skipped.

```
score = 85
if score > 50:
    print("You have passed the exam!")
```


You have passed the exam!

Here Python checks whether 85 is greater than 50. Since this is true, the indented `print` statement runs.

1.7.2 The `if...else` Statement

The `if...else` statement provides a two-way decision. If the condition is `True`, the `if` block runs; otherwise, the `else` block runs. The `else` keyword takes no condition of its own, but it still ends with a colon and is aligned with the `if`.

```
age = 16
if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are too young to vote.")
```

You are too young to vote.

1.7.3 The `if...elif...else` Statement

When there are several possibilities, the `elif` keyword (short for “else if”) is used. Python checks the conditions from top to bottom. The **first** condition that is `True` is executed, and all the rest are skipped. The final `else` acts as a default when none of the conditions are true.

Note that a **double equals** (`==`) is used to test whether two things are equal, whereas a **single equals** (`=`) only assigns a value to a variable.

```
color = "Yellow"
if color == "Red":
    print("Stop")
elif color == "Yellow":
    print("Get Ready")
elif color == "Green":
    print("Go")
else:
    print("Invalid signal color")
```

Get Ready

Python skips `"Red"` because it does not match, finds a match at `"Yellow"`, prints `Get Ready`, and ignores the remaining branches.

A useful feature of Python is that two comparisons can be **chained** together to test whether a value lies within a range. For example, $0 < W < 50$ is read exactly as in mathematics: it is `True` only when W is greater than 0 *and* less than 50.

Worked Example 1.4 — Luggage Weight Charges

An airline allows luggage up to 50 kg. Between 50 kg and 100 kg an additional charge applies, and above 100 kg a double penalty applies. This program reads the weight and decides the charge using a chained comparison.

```
W = int(input("Input your luggage weight in kg: "))
if 0 < W < 50:
    print("Weight is within the allowed limit")
elif 50 < W < 100:
    print(f"There is an additional charge for {W - 50} kg")
else:
    print(f"There is a double penalty for {W - 100} kg and normal
    ↪ additional charge for 50 kg")
```

If the user enters 72, the output is:

There is an additional charge for 22 kg

Python checks the conditions in order. The first ($0 < 72 < 50$) is false, the second ($50 < 72 < 100$) is true, so its block runs and the rest are skipped.

1.8 Control Flow: Loops

A **loop** allows a block of statements to be executed many times until a terminating condition is met. This avoids writing the same code repeatedly.

1.8.1 The while Loop

A **while** loop repeatedly executes its block of code as long as its condition remains `True`. It tests the condition *before* executing the body each time.

It is essential that something inside the loop eventually makes the condition `False`; otherwise, the loop runs forever, creating an **infinite loop**.

```
count = 1
while count <= 3:
    print(count)
    count = count + 1
print("Loop is finished!")
```

```

1
2
3
Loop is finished!

```

The variable `count` starts at 1 and increases by 1 each time. When it reaches 4, the condition `4 <= 3` becomes false and the loop stops.

1.8.2 The for Loop and the `range()` Function

A `for` loop iterates over a sequence, taking each item in turn. It is commonly used together with the built-in `range()` function, which generates a sequence of numbers.

The rule for `range(start, stop)` is that it begins at `start` and stops **one number before stop**. If only a single value is given, as in `range(4)`, the sequence starts at 0.

```

# Generates the numbers 2, 3, and 4
for i in range(2, 5):
    print(i)

# Iterating through the characters of a string
for character in "Blue":
    print(character)

```

```

2
3
4
B
l
u
e

```

Worked Example 1.7 — A Multiplication Table

A `for` loop with `range()` is a natural way to print a multiplication table. This program prints the five-times table from 1 to 10.

```

for i in range(1, 11):
    print(f"5 * {i} = {5 * i}")

```

```

5 * 1 = 5
5 * 2 = 10

```

```
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
```

Here `range(1, 11)` produces the numbers 1 to 10. On each pass the current number is stored in `i`, multiplied by 5, and displayed using an f-string.

1.8.3 Loop Control: `break`, `continue`, and `pass`

Three statements give finer control over loops:

- **`break`** terminates the loop entirely and transfers execution to the statement immediately after the loop.
- **`continue`** skips the rest of the current iteration and returns to the top of the loop to test the condition again.
- **`pass`** is a “no-operation” placeholder that does nothing. It is used where a statement is required syntactically but no action is yet needed.

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)
```

```
1
2
```

The loop prints 1 and 2; when `i` becomes 3, the `break` statement stops the loop immediately, so 3, 4, and 5 are never printed.

1.9 Interpreting Error Messages

Every programmer makes mistakes, and Python provides clear mechanisms to catch them. Errors fall into two main categories: **syntax errors** and **exceptions** (runtime errors).

1.9.1 Syntax Errors

A **syntax error**, also called a parsing error, occurs when the grammatical rules of Python are broken. The code will not run at all, and Python points an arrow to where it detected the problem. Common causes are a missing colon at the end of an `if`, `for`, or `while` line, a missing closing parenthesis, or incorrect indentation.

```
while True
    print("Hello World")
```

SyntaxError: invalid syntax

The colon after `while True` is missing.

1.9.2 Exceptions (Runtime Errors)

An **exception** occurs when the syntax is correct but an error arises while the program is running. Python produces a **traceback**, a report showing the line that caused the problem. Some common exceptions are:

- **TypeError** — an operation is applied to an inappropriate type, such as adding a number to a string;
- **ZeroDivisionError** — an attempt is made to divide by zero;
- **NameError** — a variable is used before it has been given a value.

```
result = 10 * (1 / 0)
```

ZeroDivisionError: division by zero

1.9.3 Handling Exceptions with `try...except`

Rather than allowing a program to crash, an anticipated exception can be handled using a `try...except` block. Python attempts to run the code in the `try` block; if the specified error occurs, control passes to the `except` block instead of stopping the program.

Worked Example 1.5 — Handling Bad User Input

```
try:
    number = int(input("Please enter a number: "))
    print("You entered", number)
except ValueError:
    print("Oops! That was not a valid number. Try again.")
```

If the user types `ten` instead of a digit, `int()` cannot convert it, so a `ValueError` occurs and the program prints the friendly message instead of crashing.

1.10 Importing Modules

Modules are files of pre-written, reusable Python code, such as mathematical formulas or specialised tools. Python includes a large standard library of modules. To use one, it must be loaded with an **import** statement.

1.10.1 The import Statement

The statement `import module_name` loads the entire module. To use a tool from it, the module name is written, followed by a dot (`.`), followed by the tool's name.

```
import math
root = math.sqrt(144)    # square-root function
pi_value = math.pi      # the constant pi
print(root, pi_value)
```

```
12.0 3.141592653589793
```

1.10.2 The from ... import Statement

If only a few tools from a module are needed, they may be imported directly with `from module import tool`. The module name then need not be written each time.

```
from math import pi, sqrt
print(pi)
print(sqrt(2))
```

```
3.141592653589793
1.4142135623730951
```

Writing `from math import *` copies every function from the module into the program, although importing only what is needed is generally clearer.

1.11 Function Fundamentals

A **function** is a named block of code that performs a specific task. Functions allow code to be written once and reused (“called”) as often as needed.

1.11.1 Built-in Functions

Some functions are **built-in** and always available without importing anything. Examples include `print()`, `input()`, `len()`, `abs()`, `max()`, and `min()`.

```
print(len("Statistical"))    # number of characters
print(abs(-45))              # absolute value
print(max(4, 8, 1, 9))       # largest value
print(min(4, 8, 1, 9))       # smallest value
```

```
11
45
9
1
```

1.11.2 Defining User-Defined Functions

A programmer can also create **user-defined functions** using the `def` keyword, followed by a name, parentheses, and a mandatory colon. The body of the function must be indented. A function does nothing until it is explicitly called by its name.

```
def print_welcome():
    print("Welcome to Statistical Analysis!")
    print("Let's learn Python.")

print_welcome()
```

```
Welcome to Statistical Analysis!
Let's learn Python.
```

A function such as this one, which performs an action but computes no value to return, is sometimes called a **void function**.

1.11.3 Parameters and Arguments

Functions usually need data to work with.

- **Formal parameters** are the placeholder names listed inside the parentheses in the function definition.
- **Actual arguments** are the real values passed to the function when it is called.

```
def calculate_area(length, width):    # length and width are
    ↪ parameters
    area = length * width
    print("The area is", area)

calculate_area(10, 5)                # 10 and 5 are arguments
```

The area is 50

1.11.4 Return Values

If a function should compute a value and send it back to the caller, the **return** statement is used. As soon as **return** runs, the function stops and hands back the value. If a function has no **return** statement, it automatically returns a special empty value called **None**.

```
def square_number(x):
    return x * x

result = square_number(6)    # 36 is returned and stored in result
print(result)
```

36

Python also allows a function to return **multiple values** at once, separated by commas:

```
def get_user_info():
    name = "Bob"
    age = 21
    return name, age

student_name, student_age = get_user_info()
print(student_name, student_age)
```


Bob 21

Worked Example 1.6 — A Body Mass Index Function

This example brings together user-defined functions, parameters, **return**, user input, and an f-string. The function computes the Body Mass Index (BMI) from a weight in kilograms and a height in centimetres, using the formula

$$\text{BMI} = \frac{\text{weight}}{(\text{height}/100)^2}.$$

```
def bmi(weight, height):
    bmiv = weight / ((height / 100) ** 2)
    return bmiv

h = float(input("Please enter your height in cm: "))
w = float(input("Please enter your weight in kg: "))
print(f"Your BMI is {bmi(w, h)}")
```

The function takes **weight** and **height** as parameters, computes the value, and returns it. The returned value is then displayed. Note how the height is converted from centimetres to metres inside the formula by dividing by 100.

1.11.5 Default Parameters and Keyword Arguments

Function arguments can be made flexible in two ways.

- A **default parameter** is given a value in the function header. If the caller does not supply that argument, the default is used.
- A **keyword (named) argument** is passed by writing **name=value** in the call, so the left-to-right order of the arguments does not matter.

```
def print_employee(name, department="Statistics"):    # default
    ↪ value
    print(name, "works in", department)

print_employee("Sarah", "Biology")    # overrides the default
print_employee("John")                # uses the default
```

Sarah works in Biology
John works in Statistics

```
def calculate_interest(principal, rate, time):  
    return (principal * rate * time) / 100  
  
# Because the arguments are named, their order does not matter  
interest = calculate_interest(time=2, principal=5000, rate=5)  
print(interest)
```

500.0

Practice Exercises

The following exercises can be typed and run in Google Colab.

Exercise 1.1 — Odd or Even. Write a program that asks the user to enter a whole number and prints whether it is odd or even. (*Hint: use the modulus operator %; if `val % 2 == 0`, the number is even.*)

Exercise 1.2 — Age Categorisation. Ask the user to enter an age and print whether the person is a Child (below 13), a Teenager (13–17), an Adult (18–59), or a Senior (60 or above).

Exercise 1.3 — Multiplication Table. Ask the user for a number, then use a for loop and `range()` to print the first five multiples of that number.

Exercise 1.4 — Building a Calculator. Define a function `multiply_three` that takes three parameters and returns their product. Call it with the arguments 2, 4, and 5, and print the result.

Exercise 1.5 — The Default Value. Define a function `book_flight(destination, seat_class="Economy")`. Call it once with only a destination, and a second time using keyword arguments to set the destination to "Paris" and the seat class to "First Class".

Chapter 2

Data Manipulation with Pandas

Learning Objectives

After studying this chapter, a student will be able to:

- explain what the Pandas library is and why it is used;
- create and access a one-dimensional **Series**;
- create a two-dimensional **DataFrame** and select, add, and delete its columns;
- import data from CSV and Excel files and export a DataFrame to a file;
- handle missing data and merge two DataFrames; and
- compute descriptive statistics and group data for analysis.

2.1 Introduction to Pandas

Standard Python is excellent for general programming, but it is not by itself fast or structured enough to handle large, real-world datasets such as spreadsheets with thousands of rows. For this, data scientists use an open-source add-on library called **Pandas**.

The name Pandas is derived from *panel data*, a term from econometrics for multidimensional structured datasets. Pandas provides high-performance tools for manipulating and analysing data; it may be thought of as a powerful version of a spreadsheet available directly inside Python.

Because Pandas is an external library, it must be **imported** before use. By convention it is given the short name **pd**:

```
import pandas as pd
```

Writing `as pd` means that Pandas can afterwards be referred to simply as `pd`.

A note on capital letters: Python is case-sensitive, so Pandas uses a capital **S** in **Series** and a capital **D** and **F** in **DataFrame**. Typing `pd.series()` or `pd.dataframe()` will cause an error.

2.2 The Pandas Series

Pandas has two main data structures: the **Series**, which is one-dimensional, and the **DataFrame**, which is two-dimensional. A **Series** is a one-dimensional array that can hold any single data type, such as integers, strings, or decimals.

A key feature that distinguishes a Series from an ordinary Python list is its **index** — a label attached to every value. If no index is supplied, Pandas automatically numbers the rows starting from 0. In this respect a Series behaves much like a specialised dictionary, pairing each value with a label.

2.2.1 Creating a Series from a List

```
import pandas as pd
marks = pd.Series([85, 90, 78, 92])
print(marks)
```

```
0    85
1    90
2    78
3    92
dtype: int64
```

The left-hand column (0, 1, 2, 3) is the automatically generated index; the right-hand column holds the values.

2.2.2 Creating a Series with a Custom Index

Custom text labels can be attached to the data so that each value is clearly identified:

```
import pandas as pd
age = pd.Series([18, 20, 21], index=['Parvathy', 'Archana',
    ↪ 'Arya'])
print(age)
```

```
Parvathy    18
Archana     20
Arya        21
dtype: int64
```

The second argument, `index=[...]`, replaces the default numbers 0, 1, 2 with the names.

2.2.3 Creating a Series from a Dictionary

A Python dictionary can be passed directly to `pd.Series()`. Pandas turns the dictionary's keys into the index and its values into the data:

```
import pandas as pd
data = {'Messi': 6, 'Mbappe': 5, 'Ronaldo': 2}
goal = pd.Series(data)
print(goal)
```

```
Messi       6
Mbappe      5
Ronaldo     2
dtype: int64
```

2.2.4 Accessing Data in a Series

Once a Series exists, individual values are retrieved with square brackets `[]`, using either the custom label or the underlying numerical position.

```
import pandas as pd
age = pd.Series([18, 20, 21], index=['Parvathy', 'Archana',
    ↪ 'Arya'])
print("Parvathy's age is:", age['Parvathy'])
```

```
Parvathy's age is: 18
```

Even when text labels are used, Pandas never forgets the original numerical positions, so slicing by position (for example `age[0:2]`) is still possible.

2.3 The Pandas DataFrame

While a **Series** holds a single list of values, real data is usually arranged in a table. A **DataFrame** is a two-dimensional, tabular, labelled data structure, similar to a spreadsheet or a database table. It is an ordered collection of columns, and each column may hold a different type of data.

2.3.1 Creating a DataFrame

The most common way to build a **DataFrame** is from a dictionary, in which the keys become the column names and the value lists become the columns:

```
import pandas as pd
mark = {'Name': ['Ancy', 'Sreepriya', 'Lakshmi', 'Anupama',
               ↪ 'Nivediya', 'Arya'],
        'Sem1': [85, 92, 95, 98, 50, 40],
        'Sem2': [90, 98, 92, 72, 44, 50],
        'Sem3': [91, 95, 91, 83, 98, 100],
        'Sem4': [92, 90, 93, 100, 99, 98]}
sm = pd.DataFrame(mark)
print(sm)
```

	Name	Sem1	Sem2	Sem3	Sem4
0	Ancy	85	90	91	92
1	Sreepriya	92	98	95	90
2	Lakshmi	95	92	91	93
3	Anupama	98	72	83	100
4	Nivediya	50	44	98	99
5	Arya	40	50	100	98

Pandas automatically supplies the numerical row index (0–5) on the far left.

2.3.2 Selecting Data

A single column is selected by placing its name in square brackets. A single column taken from a **DataFrame** is itself a **Pandas Series**.

```
print(sm['Name'])           # a single column
print(sm.iloc[2])          # a single row, by integer position
print(sm[:2])              # the first two rows (slicing)
```

The method `.iloc[]` (“integer location”) selects a row by its numerical position; since counting starts at 0, `sm.iloc[2]` gives the third row. Slicing with `sm[:2]` returns everything up to, but not including, row index 2.

2.3.3 Adding and Deleting Columns

A new column is created simply by assigning a list to a new column name. An existing column is removed permanently with the `del` keyword.

```
# Adding a new column
sm['Sem5'] = [96, 97, 95, 98, 99, 94]
print(sm)

# Deleting a column
del sm['Sem1']
print(sm)
```

Pandas automatically aligns the new column's values with the existing rows.

2.4 Data Operations

2.4.1 Importing and Exporting Data

Real analysts rarely type data by hand; they load files produced by other systems, most commonly **CSV** (Comma-Separated Values) files. The `pd.read_csv()` function reads a CSV file into a `DataFrame`, and the `.to_csv()` method writes a `DataFrame` back to a file.

```
import pandas as pd
# Reading a CSV file into a DataFrame
df = pd.read_csv("students.csv")
print(df)

# Exporting a DataFrame to a new CSV file, without the row index
df.to_csv("backup_students.csv", index=False)
```

The argument `index=False` prevents Pandas from writing the row numbers (0, 1, 2, ...) as an extra column in the saved file.

2.4.2 Importing Data from Excel

Because spreadsheets are so widely used, Pandas can also read Excel files with the `pd.read_excel()` function, which works much like `pd.read_csv()`. Behind the scenes Pandas uses the helper packages `openpyxl` (for newer `.xlsx` files) and `xlrd` (for older `.xls` files); these are normally already available in Google Colab.

```
import pandas as pd
# Reading the default (first) sheet
df = pd.read_excel("College Data.xlsx")
print(df.head())

# Reading a specific sheet by name
df2 = pd.read_excel("company_data.xlsx", "2000s")
print(df2.head())
```

The `.head()` method displays the first few rows so that a successful import can be confirmed; `.tail()` similarly shows the last few rows.

A practical note for Google Colab: because Colab runs in the cloud, a data file must first be **uploaded** to the session before it can be read. This is done with the folder icon on the left of the Colab window. Files uploaded this way are temporary and must be re-uploaded if the session is restarted.

2.4.3 Handling Missing Data

Real data is often incomplete: forms are left blank, and measurements fail. Pandas represents a missing value as **NaN** (Not a Number). The `numpy` library provides the official `np.nan` object used to mark data as missing.

```
import pandas as pd
import numpy as np
data = {'Student': ['Sarah', 'John', 'Mona', 'Ziad'],
        'Score': [85.0, np.nan, 90.0, np.nan]}
grades = pd.DataFrame(data)

# Option A: drop rows containing missing values
clean_grades = grades.dropna()

# Option B: fill missing values with a fixed value
filled_grades = grades.fillna(0)
```

The `.dropna()` method deletes any row that contains even a single **NaN**, whereas `.fillna(0)` replaces every **NaN** with the value supplied — here, 0.

2.4.4 Merging DataFrames

Sometimes data is split across separate tables — for example, one table of names and another of salaries. The `pd.merge()` function combines two `DataFrames` using a shared column.


```
import pandas as pd
df1 = pd.DataFrame({'Emp_ID': [101, 102, 103],
                    'Name': ['Alice', 'Bob', 'Charlie']})
df2 = pd.DataFrame({'Emp_ID': [101, 102, 104],
                    'Salary': [50000, 60000, 70000]})
merged_df = pd.merge(df1, df2, on='Emp_ID')
print(merged_df)
```

	Emp_ID	Name	Salary
0	101	Alice	50000
1	102	Bob	60000

The argument `on='Emp_ID'` tells Pandas to line up rows where the `Emp_ID` matches. By default `pd.merge()` performs an *inner merge*, keeping only rows found in **both** tables; because Charlie (103) has no salary and ID 104 has no name, both are dropped.

A related function, `pd.concat()`, stacks DataFrames on top of one another rather than joining them side by side.

2.5 Descriptive Statistics

Exploratory Data Analysis (EDA) is usually the first step when a new dataset is received. It examines the central tendency (averages) and the dispersion (spread) of the data. Pandas provides simple built-in functions for this. By default these statistical methods ignore any missing (NaN) values.

2.5.1 Basic Statistics on a Column

A single column is isolated with square brackets, and a statistic is then obtained by attaching the appropriate method:

```
print("Maximum score:", df['Score'].max())
print("Minimum age:", df['Age'].min())
print("Average score:", df['Score'].mean())
print("Standard deviation of scores:", df['Score'].std())
```

Here `.max()` finds the largest value, `.min()` the smallest, `.mean()` the arithmetic average, and `.std()` the standard deviation, which measures how spread out the values are.

2.5.2 The `.describe()` Method

Typing each statistic separately for every column is tedious. The `.describe()` method computes them all at once for every numeric column, automatically ignoring text columns:

```
print(df.describe())
```

This produces a table giving the count, mean, standard deviation, minimum, maximum, and the 25%, 50% (median), and 75% quartiles of each numeric column.

Worked Example 2.1 — Describing the College Data

Reading the course dataset and describing a single column is a common first step. The following isolates the `PlusTwo` column and summarises it.

```
import pandas as pd
cd = pd.read_excel("College Data.xlsx")
cd['SSLC'].describe()
cd['PlusTwo'].describe()
```

`.describe()` applied to the `PlusTwo` column returns its count, mean, standard deviation, minimum, quartiles, and maximum, giving an immediate summary of the students' Plus Two performance.

2.5.3 Grouping Data for Analysis

Often the average of an entire dataset is less useful than an average computed separately for each category. This is done with the **split-apply-combine** strategy: the data is *split* by a category, a calculation is *applied* to each group, and the results are *combined* into a table. The `.groupby()` method performs this.

```
market_data = pd.DataFrame({
    'City': ['Dubai', 'Abu Dhabi', 'Dubai', 'Sharjah', 'Abu
    ↪ Dhabi'],
    'Sales': [200, 150, 300, 120, 180]
})
grouped_sales = market_data.groupby('City')['Sales'].sum()
print(grouped_sales)
```

In the final line, `.groupby('City')` splits the table into groups by city, `['Sales']` selects the column to work on, and `.sum()` adds the values within each group. The result shows the total sales for each city separately.

Practice Exercises

Exercise 2.1 — Your First Series. Create a Pandas Series containing the numbers 10, 20, 30, 40, 50 with the default numerical index, and print it.

Exercise 2.2 — Dictionary to Series. Build a dictionary of three countries and their capitals (for example, 'Japan': 'Tokyo'), convert it to a Series, and print the capital of Japan using its index label.

Exercise 2.3 — Create a Table. Create a DataFrame called `inventory` from a dictionary with keys 'Product' (Laptop, Mouse, Keyboard) and 'Price' (1000, 20, 50), and print it.

Exercise 2.4 — Grouping Data. Create a DataFrame with columns 'Department' (IT, HR, IT, HR) and 'Salary' (some values), then use `.groupby()` to find the average salary for each department.

Chapter 3

Data Visualization

Learning Objectives

After studying this chapter, a student will be able to:

- describe the main Python visualization libraries;
- create line plots, bar plots, pie charts, and histograms with Matplotlib;
- prepare a real dataset for plotting using the Pandas `.map()` method;
- create box plots, strip plots, and swarm plots with Seaborn; and
- create scatter plots, heatmaps, density plots, cumulative-frequency plots, and error bars.

Data visualization is the process of interpreting data in a pictorial or graphical form. A single well-chosen chart can communicate the message of thousands of numbers far more effectively than a table.

Python has several visualization libraries. **Matplotlib** is the foundational library for basic charts; its `pyplot` module makes plotting straightforward and is conventionally imported as `plt`. **Seaborn** is built on top of Matplotlib and produces attractive statistical graphics with less code; it is imported as `sns`. Other libraries mentioned in the syllabus include **Plotly** (for interactive web charts), **ggplot** (which builds charts layer by layer using the “grammar of graphics”), **Geoplotlib** (for plotting geographical map data), and the plotting tools built directly into **Pandas**.

The examples in this chapter use the course dataset, `College_Data.csv`, described in the Preface.

3.1 Basic Charts with Matplotlib

Every Matplotlib chart is displayed by calling `plt.show()` at the end. Titles and axis labels are added with `plt.title()`, `plt.xlabel()`, and `plt.ylabel()`.

3.1.1 Line Plot

A **line plot** shows how a quantity changes across a continuous sequence, such as values over time. The two sequences passed to `plt.plot()` — the x-coordinates and the y-coordinates — must be of the same length.

```
import matplotlib.pyplot as plt
years = [2018, 2019, 2020, 2021, 2022]
sales = [120, 135, 150, 145, 170]
plt.plot(years, sales)
plt.title("Company Sales Over Time")
plt.xlabel("Year")
plt.ylabel("Sales in Thousands")
plt.show()
```

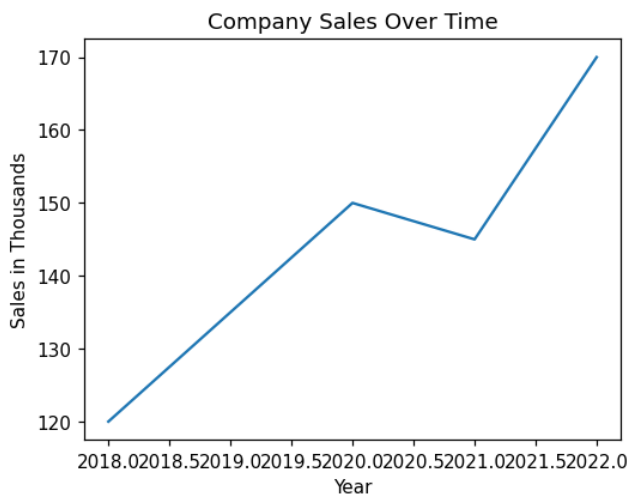


Figure 3.1: A line plot of sales against year.

3.1.2 Bar Plot

A **bar plot** compares the quantities of distinct categories. The function `plt.bar()` draws vertical rectangles, and the `color` argument changes their

colour.

```
import matplotlib.pyplot as plt
products = ['Apples', 'Oranges', 'Bananas']
quantities = [50, 30, 45]
plt.bar(products, quantities, color='orange')
plt.title("Fruit Inventory")
plt.show()
```

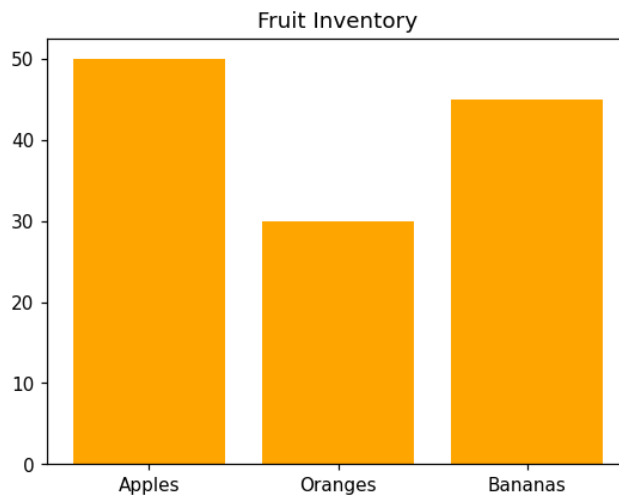


Figure 3.2: A bar plot comparing fruit quantities.

3.1.3 Pie Chart

A **pie chart** shows how a whole is divided into proportional parts. In `plt.pie()`, the numerical data are given first and the category names are supplied through the `labels` argument. The `autopct='%1.1f%%'` argument tells Python to calculate each slice's percentage and display it to one decimal place.

```
import matplotlib.pyplot as plt
expenses = ['Rent', 'Food', 'Transport']
costs = [1200, 800, 400]
plt.pie(costs, labels=expenses, autopct='%1.1f%%')
plt.title("Monthly Budget")
plt.show()
```

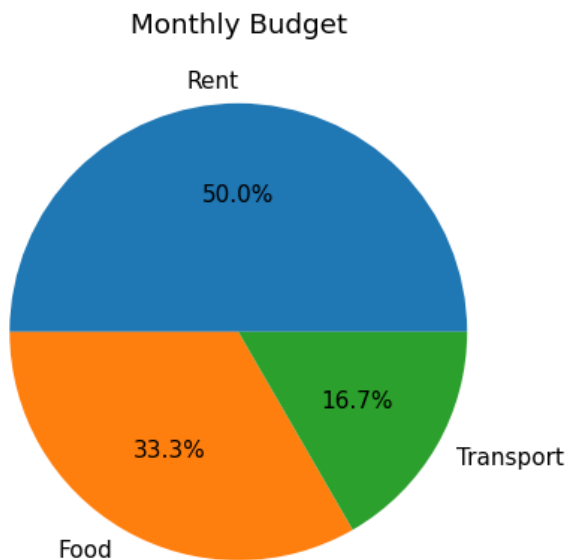


Figure 3.3: A pie chart of a monthly budget.

3.1.4 Histogram

A **histogram** looks like a bar chart but serves a different purpose: it groups a list of numbers into ranges called **bins** and counts how many values fall into each, revealing the distribution of the data. It takes a single list of numbers rather than an x and a y list.

```
import matplotlib.pyplot as plt
scores = [55, 62, 67, 71, 73, 74, 78, 80, 82, 85, 88, 90, 92, 95]
plt.hist(scores, bins=4, edgecolor='black')
plt.title("Distribution of Exam Scores")
plt.xlabel("Score Ranges")
plt.ylabel("Number of Students")
plt.show()
```

The `bins=4` argument divides the data into four equal ranges, and `edgecolor='black'` draws a border around each bar.

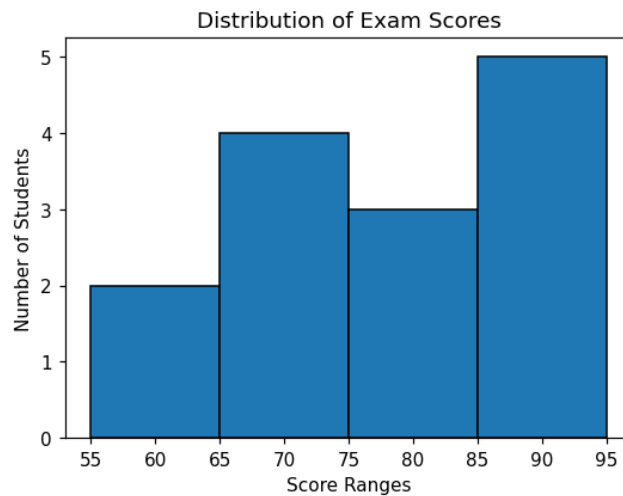


Figure 3.4: A histogram of exam scores.

3.2 Preparing the College Data for Plotting

The categorical variables in the College Data (such as **Gender**, **Locality**, and **College**) are stored as numbers. If plotted directly, the charts would show 0 and 1 on the labels, which is confusing. The Pandas `.map()` method replaces these codes with readable text before plotting. It works like a dictionary lookup, scanning a column and replacing each code with its label.

```
import pandas as pd
import matplotlib.pyplot as plt
cd = pd.read_excel("College Data.xlsx")
cd['Gender'] = cd['Gender'].map({0: 'Male', 1: 'Female'})
cd['Locality'] = cd['Locality'].map({0: 'Urban', 1: 'Rural'})
cd['College'] = cd['College'].map({1: 'College A', 2: 'College
↵ B', 3: 'College C'})
```

The examples that follow assume the data has been read and cleaned in this way.

3.2.1 Pie Chart — Gender Distribution

To chart a category, its values are first counted with `.value_counts()`.

```
gender_counts = cd['Gender'].value_counts()
plt.pie(gender_counts.values, labels=gender_counts.index,
        ↪ autopct='%1.1f%%', startangle=90)
plt.title("Gender Distribution of PG Students")
plt.show()
```

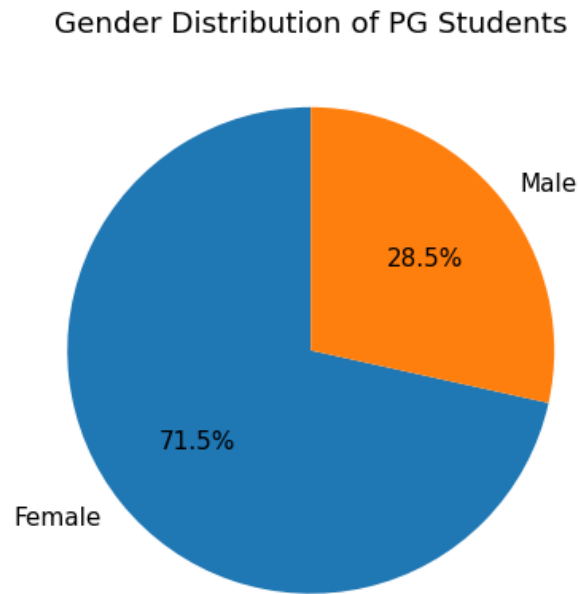


Figure 3.5: Pie chart of gender distribution in the College Data.

Here `startangle=90` rotates the pie so that it appears upright.

3.2.2 Bar Plot — Students per College

```
college_counts = cd['College'].value_counts()
plt.bar(college_counts.index, college_counts.values,
        ↪ color='orange', edgecolor='black')
plt.title("Number of Students in Each College")
plt.xlabel("College Name")
plt.ylabel("Number of Students")
plt.show()
```

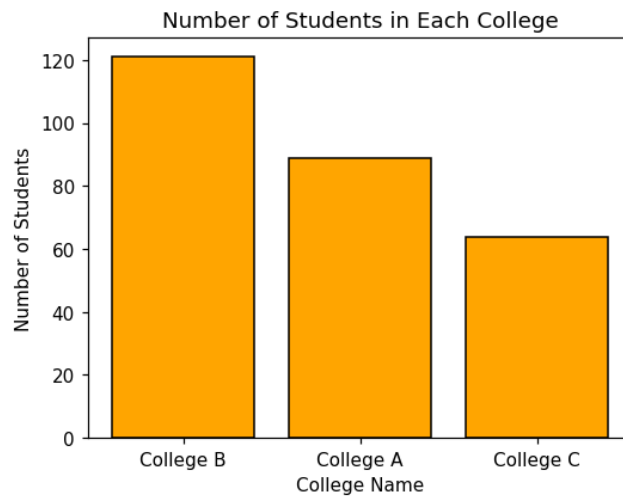


Figure 3.6: Bar plot of students per college.

3.2.3 Histogram — Distribution of Degree Marks

For a histogram the raw column of marks is passed directly, without counting first.

```
plt.hist(cd['Degree'], bins=10, color='skyblue',
        ↪ edgecolor='black')
plt.title("Distribution of Degree Marks")
plt.xlabel("Percentage of Marks")
plt.ylabel("Number of Students")
plt.show()
```

3.2.4 Line Plot — Average Marks Progression

Although this dataset has no year column, a line plot can show how the average mark changes across the three educational stages SSLC, Plus Two, and Degree.

```
avg_sslc = cd['SSLC'].mean()
avg_plustwo = cd['PlusTwo'].mean()
avg_degree = cd['Degree'].mean()
education_stages = ['SSLC', 'PlusTwo', 'Degree']
average_marks = [avg_sslc, avg_plustwo, avg_degree]
plt.plot(education_stages, average_marks, marker='o',
        ↪ color='red', linestyle='--')
```

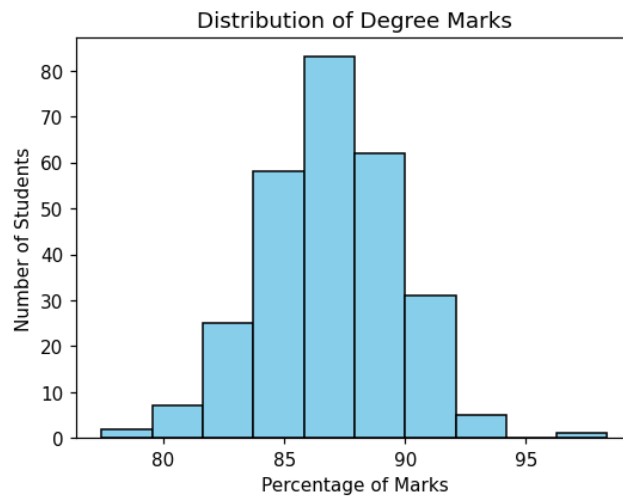


Figure 3.7: Histogram of degree marks.

```
plt.title("Trend of Average Marks by Education Stage")
plt.xlabel("Education Stage")
plt.ylabel("Average Marks (%)")
plt.grid(True)
plt.show()
```

Here `marker='o'` places a dot at each point, `linestyle='--'` makes the line dashed, and `plt.grid(True)` adds a background grid for easier reading.

3.3 Statistical Charts with Seaborn

Seaborn is imported alongside Matplotlib and Pandas. The data is loaded and cleaned in the same way as before.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

3.3.1 Box Plot

A **box plot** (box-and-whisker plot) displays the distribution of data through a five-number summary: minimum, first quartile, median, third quartile, and

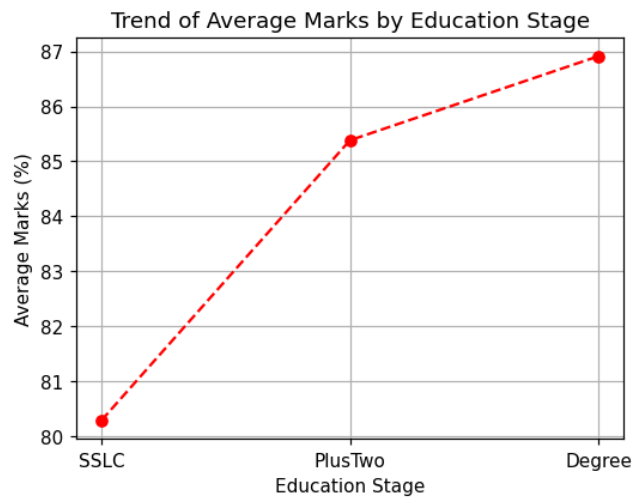


Figure 3.8: Line plot of average marks by stage.

maximum. It is the best chart for spotting **outliers**. In Seaborn the categorical variable is given to **x**, the numerical variable to **y**, and the DataFrame to **data**.

```
sns.boxplot(x='College', y='Degree', data=cd)
plt.title("Box Plot of Degree Marks by College")
plt.xlabel("College Name")
plt.ylabel("Degree Marks (%)")
plt.show()
```

The box represents the middle 50% of students, the line inside it is the median, the whiskers show the rest of the distribution, and any isolated dots beyond the whiskers are outliers.

3.3.2 Strip Plot

A **strip plot** is a scatter plot in which one axis represents categories. The `jitter=True` argument spreads overlapping points slightly sideways so that individual points remain visible even when several students share the same mark.

```
sns.stripplot(x='Gender', y='PlusTwo', data=cd, jitter=True)
plt.title("Strip Plot of PlusTwo Marks by Gender")
plt.xlabel("Student Gender")
plt.ylabel("PlusTwo Marks (%)")
plt.show()
```

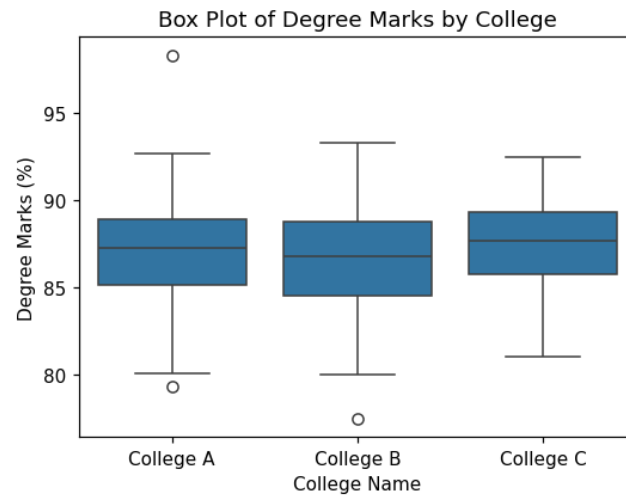


Figure 3.9: Box plot of degree marks by college.

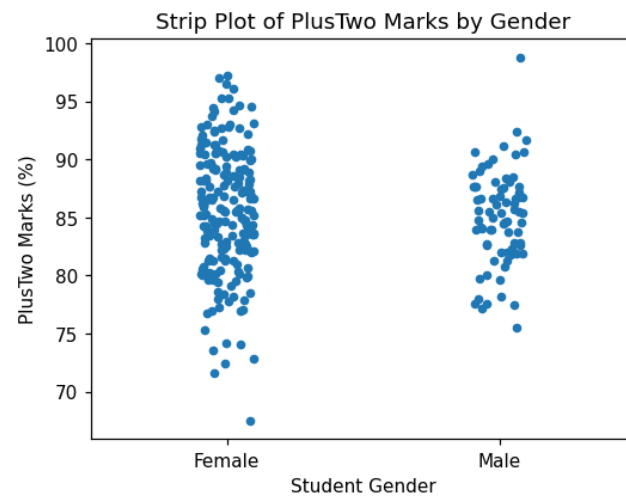


Figure 3.10: Strip plot of PlusTwo marks by gender.

3.3.3 Swarm Plot

Even with jitter, a strip plot can look crowded. A **swarm plot** positions every point so that none overlap, producing a shape resembling a swarm of bees; its width shows where the data is most dense.

```
sns.swarmplot(x='Locality', y='SSLC', data=cd)
plt.title("Swarm Plot of SSLC Marks by Locality")
plt.xlabel("Student Locality")
plt.ylabel("SSLC Marks (%)")
plt.show()
```

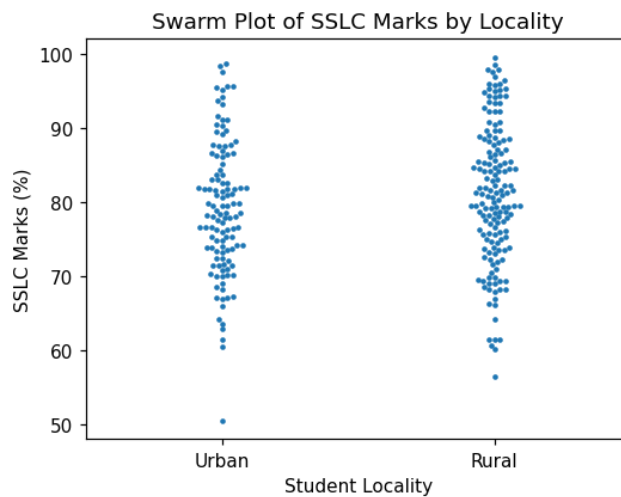


Figure 3.11: Swarm plot of SSLC marks by locality.

3.4 Advanced Plots

3.4.1 Scatter Plot

A **scatter plot** shows the relationship between two numerical variables. Here it is used to see whether students who scored well in Plus Two also scored well in their Degree.

```
plt.scatter(cd['PlusTwo'], cd['Degree'])
plt.xlabel("PlusTwo Marks (%)")
plt.ylabel("Degree Marks (%)")
```

```
plt.title("Scatter Plot of PlusTwo vs Degree Marks")
plt.show()
```

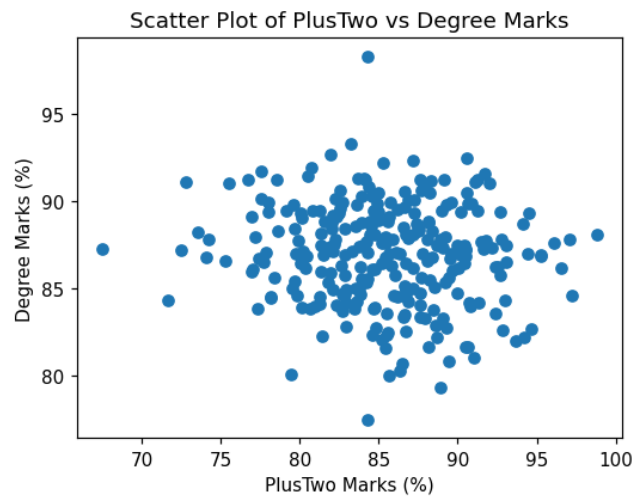


Figure 3.12: Scatter plot of PlusTwo against Degree marks.

The same relationship can be drawn interactively with **Plotly**, or layer by layer with **ggplot**:

```
# Interactive scatter plot with Plotly (displays inside Colab)
from plotly.offline import init_notebook_mode, iplot
import plotly.graph_objs as go
init_notebook_mode(connected=True)
trace = go.Scatter(x=cd['PlusTwo'], y=cd['Degree'],
    ↪ mode='markers')
iplot([trace])
```

```
# Scatter plot with ggplot (requires: pip install ggplot)
from ggplot import *
p = ggplot(aes(x='PlusTwo', y='Degree', color='Gender'), data=cd)
p + geom_point() + ggtitle("PlusTwo vs Degree Marks")
```

In the Plotly code, `mode='markers'` draws dots rather than a line, and `iplot()` produces an interactive chart on which values appear when the mouse hovers over a point. In the ggplot code, `aes()` maps the axes and colour, and `geom_point()` adds the layer of dots.

3.4.2 Heatmap

A **heatmap** uses colour to show the intensity of the numbers in a two-dimensional matrix. It is an excellent way to display the correlation between variables. The Pandas `.corr()` method computes the correlations first.

```
correlations = cd[['SSLC', 'PlusTwo', 'Degree']].corr()
sns.heatmap(correlations, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap of Student Marks")
plt.show()
```

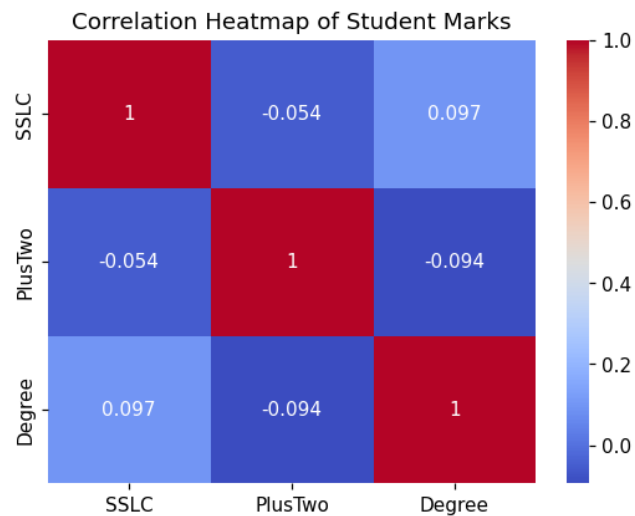


Figure 3.13: Correlation heatmap of the three mark columns.

The argument `annot=True` writes the correlation values inside the coloured cells, and `cmap='coolwarm'` shades high values red and low values blue.

3.4.3 Density Plot and Cumulative Frequency

A **density plot** is a smoothed version of a histogram showing the shape of the data, and can be drawn directly from a Pandas column. A **cumulative-frequency** plot shows a running total, obtained by adding `cumulative=True` to a histogram.

```
# Density plot using Pandas directly
cd['Degree'].plot.density(color='purple', linewidth=3)
plt.title("Density Plot of Degree Marks")
```

```
plt.xlabel("Marks (%)")
plt.show()
```

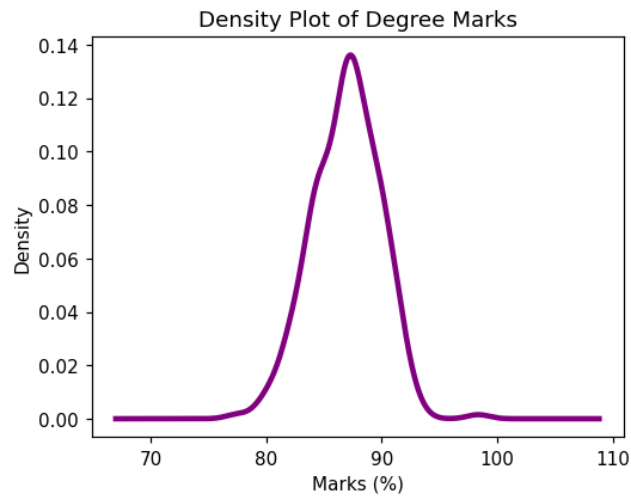


Figure 3.14: Density plot of degree marks.

```
# Cumulative frequency using Matplotlib
plt.hist(cd['Degree'], bins=20, cumulative=True, color='orange',
        ↪ edgecolor='black')
plt.title("Cumulative Frequency of Degree Marks")
plt.xlabel("Marks (%)")
plt.ylabel("Running Total of Students")
plt.show()
```

3.4.4 Error Bars

An **error bar** is a line drawn through a data point to represent uncertainty or variation. Here the average Degree mark for each college is plotted, with error bars showing the standard deviation.

```
avg_marks = cd.groupby('College')['Degree'].mean()
std_marks = cd.groupby('College')['Degree'].std()
plt.bar(avg_marks.index, avg_marks.values, yerr=std_marks.values,
        capsize=5, color='lightblue', edgecolor='black')
plt.title("Average Degree Marks per College with Error Bars")
```

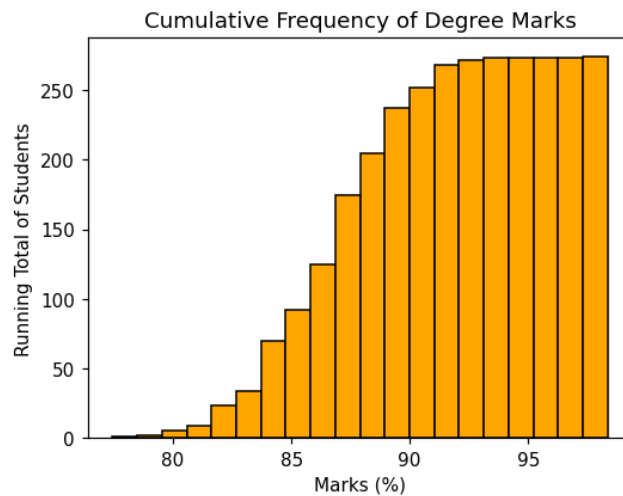


Figure 3.15: Cumulative frequency plot of degree marks.

```
plt.ylabel("Average Marks (%)")
plt.show()
```

The `.groupby()` method finds the mean and standard deviation for each college. The argument `yerr=std_marks.values` draws the vertical error lines, and `capsize=5` adds small horizontal caps at their ends.

Practice Exercises

Exercise 3.1 — A Line Plot. Heart rate was recorded each hour for four hours as [72, 75, 78, 74] at hours [1, 2, 3, 4]. Draw a line plot of this data with a title.

Exercise 3.2 — A Bar Plot. Three departments ['IT', 'HR', 'Sales'] have [10, 6, 14] employees. Draw a green bar plot comparing them.

Exercise 3.3 — A Histogram. Using the College Data, draw a histogram of the SSLC column with 20 bins and a black edge colour.

Exercise 3.4 — A Box Plot. Using the College Data, draw a Seaborn box plot of Degree marks grouped by Gender.

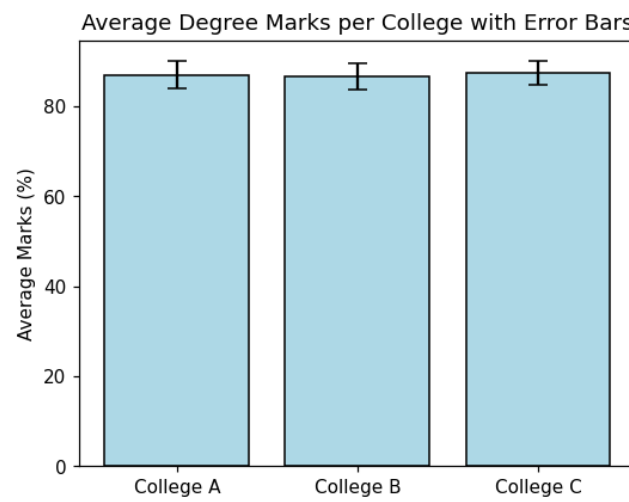


Figure 3.16: Bar chart of average degree marks per college with error bars.

Chapter 4

Statistical Data Analysis

Learning Objectives

After studying this chapter, a student will be able to:

- generate simulated data from the uniform and normal distributions;
- explain hypothesis testing using the null and alternative hypotheses and the p-value;
- test whether data is normally distributed using the Shapiro–Wilk test;
- carry out one-sample, two-sample (independent), and paired tests, choosing a parametric or non-parametric test as appropriate;
- carry out one-way and two-way ANOVA to compare three or more groups;
- measure the relationship between two numerical variables using correlation; and
- build and interpret simple linear, multiple linear, and logistic regression models.

This chapter uses the **statsmodels** and **scipy** libraries for statistical testing, together with **numpy** for generating data. All examples use the course dataset, **College_Data.csv**. Rather than treating statistics as a branch of mathematics, this chapter treats it as a logical way of using data to answer real questions.

4.1 Generating Simulated Data

Sometimes we do not yet have real data but want to imagine, or **simulate**, a situation. For this we use the **numpy** library, conventionally imported as **np**. Two ways in which data can be spread out are especially important: the uniform distribution and the normal distribution.

4.1.1 The Uniform Distribution

In a **uniform distribution** every value in a range has an equal chance of being chosen, like rolling a fair die where each of the six faces is equally likely.

```
import numpy as np
import matplotlib.pyplot as plt

# Generate 1,000 random decimal numbers between 1 and 100
uniform_scores = np.random.uniform(low=1.0, high=100.0,
    ↪ size=1000)

plt.hist(uniform_scores, bins=10, color='lightgreen',
    ↪ edgecolor='black')
plt.title("Uniform Distribution (Equal Chance)")
plt.xlabel("Generated Values")
plt.ylabel("How Often They Appeared")
plt.show()
```

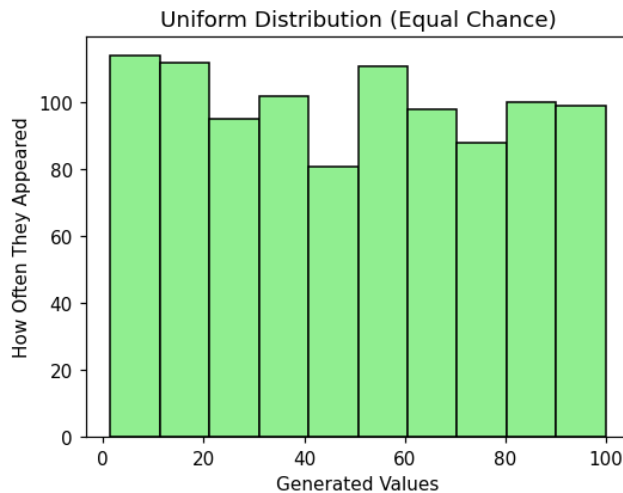


Figure 4.1: A uniform distribution produces a flat, box-like histogram.

The function `np.random.uniform(low=1.0, high=100.0, size=1000)` asks Python to generate 1,000 numbers between 1 and 100, each equally likely. Because every value is equally likely, the bars of the histogram are all roughly the same height, giving a flat shape.

4.1.2 The Normal Distribution

In real life, quantities such as height, weight, or exam marks are not equally likely. Most people cluster near the average, and very few are extremely high or extremely low. This forms a symmetrical **bell curve**, called the **normal distribution**.

```
import numpy as np
import matplotlib.pyplot as plt

# Lock the random numbers so they are the same every time
np.random.seed(42)

# Generate 1,000 scores with an average of 75 and a spread of 10
bell_curve_scores = np.random.normal(loc=75.0, scale=10.0,
    ↪ size=1000)

plt.hist(bell_curve_scores, bins=15, color='skyblue',
    ↪ edgecolor='black')
plt.title("Normal Distribution (Bell Curve)")
plt.xlabel("Simulated Exam Marks")
plt.ylabel("How Often They Appeared")
plt.show()
```

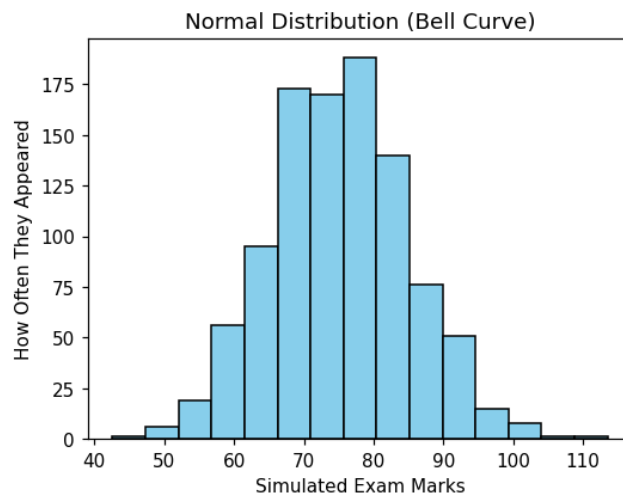


Figure 4.2: A normal distribution produces a symmetrical, hill-shaped histogram.

Two details are worth noting. `np.random.seed(42)` fixes the starting point of the random-number generator, so that the same numbers are produced every

time the code is run — useful when everyone in a class should get identical results. In `np.random.normal(loc=75.0, scale=10.0, size=1000)`, `loc` is the average (mean) around which the numbers cluster, and `scale` is the standard deviation, which controls how spread out they are. The peak of the resulting hill sits at 75, showing that most simulated students scored near the average.

4.2 Hypothesis Testing: The Courtroom Analogy

Most statistical questions are answered by **hypothesis testing**. The logic is exactly like a criminal trial.

A **hypothesis** is an educated guess. Every test involves two competing hypotheses:

- The **null hypothesis** (H_0) is the *innocent defendant*. It is the default assumption that nothing special is happening: there is no real difference or effect, and any pattern we see is just coincidence.
- The **alternative hypothesis** (H_1) is the *prosecutor's claim*. It states that something real is happening: a genuine difference or effect exists.

Our **data** is the evidence presented at the trial.

The **p-value** is the “doubt meter”. It measures the probability of seeing evidence like ours purely by chance, *if the defendant were actually innocent* (that is, if H_0 were true). The standard threshold is $\alpha = 0.05$, which plays the role of “beyond reasonable doubt”:

- If **p < 0.05** (low doubt): the chance that this happened by pure coincidence is less than 5%. We **reject** H_0 and conclude H_1 — something real is happening.
- If **p ≥ 0.05** (high doubt): there is too much doubt to convict. We **accept** H_0 — there is not enough evidence of a real effect.

This single rule — compare the p-value to 0.05 — is used in every test in this chapter.

4.3 Checking for Normality: The Gatekeeper

Before choosing a test, we must run a diagnostic check on our data: does the column of numbers follow a normal bell curve or not? The answer decides which kind of test we may use.

- If the data **is** normal, we use precise tools called **parametric tests**.
- If the data is **not** normal, we use sturdier backup tools called **non-parametric tests**.

The standard check is the **Shapiro–Wilk test**. Set up as a courtroom trial:

- H_0 (innocent): the data *is* normally distributed.
- H_1 (guilty): the data is *not* normally distributed.

```
import pandas as pd
import scipy.stats as stats

df = pd.read_excel('college_data.xlsx')

# Run the Shapiro-Wilk test on the Degree marks
stat, p_value = stats.shapiro(df['Degree'])
print("Normality Test Result:")
print("p-value:", p_value)
```

The function `stats.shapiro(df['Degree'])` returns two numbers: a test statistic and a p-value. We store the p-value and read it using the standard rule. If the p-value is **greater than 0.05**, we accept H_0 : the data is normal, and a parametric test may be used. If it is **less than 0.05**, we reject H_0 : the data is not normal, and a non-parametric test is required.

Note the way the two returned values are captured: writing `stat, p_value = stats.shapiro(...)` unpacks them into two separate variables, so that the p-value can be printed or formatted on its own.

4.4 One-Sample Tests

A **one-sample test** compares the average of a single group against a fixed number. Consider the question: “*Is the average Degree mark in our college equal to 86%?*”

- H_0 : the average Degree mark is equal to 86%.
- H_1 : the average Degree mark is not equal to 86%.

Which test we use depends on the normality check from the previous section.

Worked Example 4.1 — One-Sample t-test (data is normal)

If the Shapiro–Wilk p-value for **Degree** was greater than 0.05, the data is normal and we use the parametric **one-sample t-test**.

```
t_stat, p_val = stats.ttest_1samp(df['Degree'], popmean=86.0)
print("t-test Result:")
print("p-value:", p_val)
```

`stats.ttest_1samp(df['Degree'], popmean=86.0)` checks whether the average of the Degree column is close enough to the hypothesised value 86.0. If `p_val` is less than 0.05, we reject H_0 and conclude the average is *not* 86%. If it is greater than 0.05, we accept H_0 and conclude the average *is* 86%.

Worked Example 4.2 — Wilcoxon Signed-Rank test (data is not normal)

If the normality p-value was below 0.05, the data is skewed and the t-test cannot be trusted. We use the non-parametric **Wilcoxon signed-rank test**, which works on the median instead of the mean.

- H_0 : the median Degree mark is equal to 86%.
- H_1 : the median Degree mark is not equal to 86%.

The Wilcoxon test needs the hypothesised value subtracted from the column first.

```
# Subtract the hypothesised value from the column
difference_from_hypothesised = df['Degree'] - 86.0

wilc_stat, p_val = stats.wilcoxon(difference_from_hypothesised)
print("Wilcoxon Test Result:")
print("p-value:", p_val)
```

As always, if `p_val` is less than 0.05 we reject H_0 and conclude the marks differ significantly from 86%.

4.5 Two-Sample Tests: Comparing Two Separate Groups

A **two-sample (independent) test** compares two completely separate groups. Consider: “Do male and female students get different average Degree marks?”

It helps to look at a boxplot first, to form a visual expectation. If the two boxes sit at clearly different heights, we expect a low p-value; if they overlap heavily, we expect a high one.

```

import pandas as pd
import scipy.stats as stats
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_excel('college_data.xlsx')
df['Gender'] = df['Gender'].map({0: 'Male', 1: 'Female'})

sns.boxplot(x='Gender', y='Degree', data=df)
plt.title("Degree Marks by Gender")
plt.show()

```

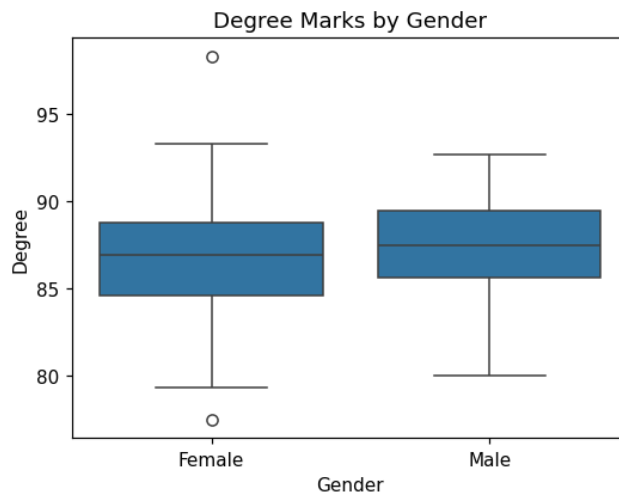


Figure 4.3: Boxplot comparing Degree marks for male and female students.

Next we check normality for **each** group separately, then choose the test.

```

males = df[df['Gender'] == 'Male']['Degree']
females = df[df['Gender'] == 'Female']['Degree']

_, p_male = stats.shapiro(males)
_, p_female = stats.shapiro(females)
print(f"Male Normality p-value: {p_male:.4f}")
print(f"Female Normality p-value: {p_female:.4f}")

```

Here `df[df['Gender'] == 'Male']['Degree']` picks out just the Degree marks of the male students, and similarly for females. The underscore in `_, p_male = stats.shapiro(males)` is a placeholder that discards the test statistic, keeping only the p-value.

This is the **fork in the road**:

- If **both** p-values are above 0.05, both groups are normal, and we use the parametric **independent t-test**.
- If **either** p-value is below 0.05, we use the non-parametric **Mann–Whitney U test**.

```
# If both groups are normal: independent t-test
t_stat, p_val_t = stats.ttest_ind(males, females)
print(f"t-test p-value: {p_val_t:.4f}")

# If the data is skewed: Mann-Whitney U test
u_stat, p_val_u = stats.mannwhitneyu(males, females)
print(f"Mann-Whitney p-value: {p_val_u:.4f}")
```

`stats.ttest_ind(males, females)` compares the two averages; `stats.mannwhitneyu(males, females)` does the same job using ranks rather than averages, which is why it is safe for skewed data. Whichever test is appropriate, the decision rule is unchanged: p below 0.05 means a significant difference.

4.6 Paired Tests: Comparing the Same People Twice

A **paired test** compares the *same* individuals measured at two different times, rather than two separate groups. Consider: “*Did students’ marks change significantly from PlusTwo to Degree?*” Because each student appears in both columns, the two measurements are locked together in pairs.

For a paired test there is an important difference in the normality check: we do **not** test each column separately. Instead we test the normality of the **difference** between the two columns. Throughout this section the difference is taken as *later minus earlier*, so a positive difference means improvement.

```
# Check the normality of the DIFFERENCE between the two scores
differences = df['Degree'] - df['PlusTwo']
_, p_norm = stats.shapiro(differences)
print(f"Differences Normality p-value: {p_norm:.4f}")
```

Then the appropriate paired test is chosen:

```
# If the differences are normal: paired t-test
t_paired, p_paired = stats.ttest_rel(df['Degree'], df['PlusTwo'])
```

```
print(f"Paired t-test p-value: {p_paired:.4f}")

# If the differences are not normal: Wilcoxon signed-rank test
wilc_stat, p_wilc = stats.wilcoxon(df['Degree'], df['PlusTwo'])
print(f"Wilcoxon Paired p-value: {p_wilc:.4f}")
```

`stats.ttest_rel()` stands for “related t-test”; it pairs the first Degree mark with the first PlusTwo mark, the second with the second, and so on. `stats.wilcoxon()` is its non-parametric backup. To keep the direction consistent with the normality check, the columns are listed in the same order (Degree first, PlusTwo second).

4.7 Comparing Three or More Groups: ANOVA

When there are three or more groups, running many two-sample t-tests is poor practice, because each extra test raises the chance of a false positive. Instead we use **ANOVA (Analysis of Variance)**, which compares all the groups at once. Intuitively, ANOVA checks whether the differences *between* the groups are much larger than the natural variation *inside* each group.

4.7.1 One-Way ANOVA

One-way ANOVA compares groups defined by a single factor. Consider: “Do students from College A, College B, and College C get different Degree marks?”

```
import pandas as pd
import scipy.stats as stats
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_excel('college_data.xlsx')
df['College'] = df['College'].map({1: 'College A', 2: 'College
    ↪ B', 3: 'College C'})

sns.boxplot(x='College', y='Degree', data=df)
plt.title("Degree Marks by College")
plt.show()

# Separate the data into three groups
g_a = df[df['College'] == 'College A']['Degree']
g_b = df[df['College'] == 'College B']['Degree']
g_c = df[df['College'] == 'College C']['Degree']
```

```
# Check normality for all three groups
_, p_a = stats.shapiro(g_a)
_, p_b = stats.shapiro(g_b)
_, p_c = stats.shapiro(g_c)
print(f"College A: {p_a:.4f}")
print(f"College B: {p_b:.4f}")
print(f"College C: {p_c:.4f}")
```

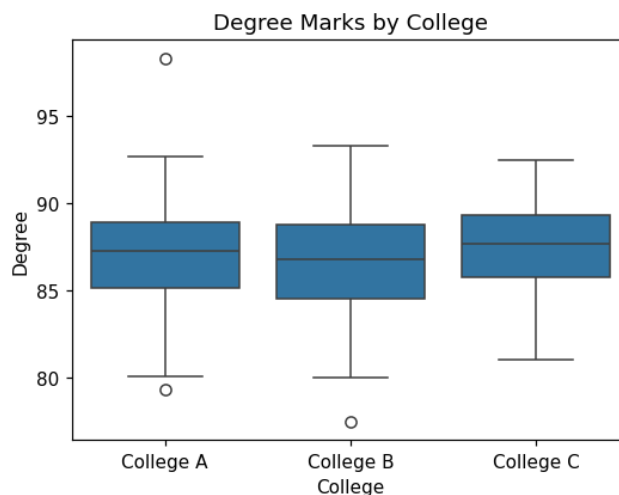


Figure 4.4: Boxplot comparing Degree marks across three colleges.

Note that each Shapiro–Wilk result is unpacked into its own p-value (`_, p_a = stats.shapiro(g_a)`) before formatting, since the test returns two values and only the p-value is needed. The fork is the same as before:

```
# If all groups are normal: one-way ANOVA
f_val, p_anova = stats.f_oneway(g_a, g_b, g_c)
print(f"One-way ANOVA p-value: {p_anova:.4f}")

# If at least one group is not normal: Kruskal-Wallis test
h_val, p_kruskal = stats.kruskal(g_a, g_b, g_c)
print(f"Kruskal-Wallis p-value: {p_kruskal:.4f}")
```

`stats.f_oneway()` is the parametric one-way ANOVA; `stats.kruskal()` is its non-parametric equivalent, the three-or-more-group version of the Mann–Whitney U test. A p-value below 0.05 means at least one college differs significantly from the others.

4.7.2 Two-Way ANOVA

Two-way ANOVA examines *two* factors at the same time, and also whether they interact. Consider: “Does the *Degree* mark depend on *Gender* and on *Locality*, and do the two factors combine in any special way?” For this we use the **statsmodels** library, which can fit a model with more than one factor.

```
import statsmodels.api as sm
import statsmodels.formula.api as smf

df = pd.read_excel('college_data.xlsx')
df['Gender'] = df['Gender'].map({0: 'Male', 1: 'Female'})
df['Locality'] = df['Locality'].map({0: 'Urban', 1: 'Rural'})

sns.boxplot(x='Gender', y='Degree', hue='Locality', data=df)
plt.title("Degree Marks by Gender and Locality")
plt.show()

# Fit a two-way model: Degree explained by Gender and Locality
model = smf.ols('Degree ~ C(Gender) + C(Locality)',
    ↪ data=df).fit()

# Produce the ANOVA table
anova_table = sm.stats.anova_lm(model, typ=2)
print(anova_table)
```

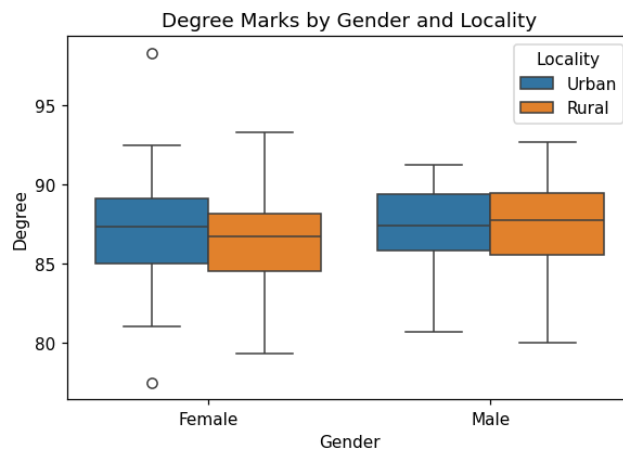


Figure 4.5: Boxplot of Degree marks split by both Gender and Locality.

In the formula `'Degree ~ C(Gender) + C(Locality)'`, the part before the tilde (`~`) is the quantity being explained (`Degree`), and the parts after it are the

two factors. Wrapping each factor in `C(...)` tells statsmodels to treat it as a category rather than a number. The function `anova_lm` then prints a table with one row per factor, each with its own p-value. Reading is exactly as before: a factor whose p-value is below 0.05 has a significant effect on the Degree mark.

4.8 Correlation

Correlation measures how strongly two numerical columns move together. It is summarised by a single number, the correlation coefficient r , which always lies between -1 and $+1$:

- **Positive** (r near $+1$): as one value rises, the other rises too (for example, hours studied and exam score).
- **Negative** (r near -1): as one rises, the other falls (for example, hours spent gaming and exam score).
- **Zero** (r near 0): there is no relationship (for example, shoe size and exam score).

Before calculating anything, it is best to look at a **scatter plot**.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_excel('college_data.xlsx')

plt.scatter(df['PlusTwo'], df['Degree'], color='purple',
            ↪ alpha=0.7)
plt.title("Do PlusTwo Marks and Degree Marks Move Together?")
plt.xlabel("PlusTwo Marks (%)")
plt.ylabel("Degree Marks (%)")
plt.grid(True)
plt.show()
```

Pandas can compute the correlation between several columns at once with the `.corr()` method.

```
correlation_matrix = df[['SSLC', 'PlusTwo', 'Degree']].corr()
print("Correlation Table (r-values):")
print(correlation_matrix)
```

The result is a grid showing how each column correlates with every other column. Every column always has a correlation of exactly 1.0 with itself, along the diagonal. To read any other cell, look at the number: a value near $+1$ is a strong positive connection, a value near -1 a strong negative connection, and a value near 0 means the two marks are essentially unrelated.

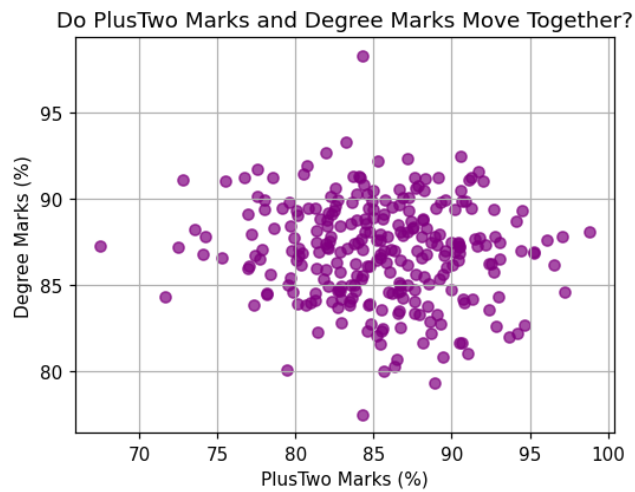


Figure 4.6: Scatter plot of PlusTwo marks against Degree marks.

4.9 Simple Linear Regression

Correlation tells us *whether* two variables are related. **Regression** goes further and gives an actual equation to **predict** one variable from another. Just as one might predict a house's price from its size by drawing a straight trend line through past sales, regression finds the exact equation of that line:

$$\text{Degree Mark} = (\text{Slope} \times \text{PlusTwo Mark}) + \text{Starting Point}.$$

The **slope** says how much the Degree mark is predicted to rise for each one-percent rise in PlusTwo, and the **starting point** (intercept) is the predicted Degree mark for a student who scored zero in PlusTwo. We fit the model with **statsmodels**.

```
import statsmodels.formula.api as smf

# Predict 'Degree' (dependent) from 'PlusTwo' (independent)
model = smf.ols('Degree ~ PlusTwo', data=df).fit()

print(model.summary())
```

The `summary()` output is a large table, but a beginner needs only three values from it:

1. **R-squared** (top right) is the model's accuracy score. An R-squared of 0.64 would mean the model explains 64% of the variation in Degree marks; higher is better.

2. The **Intercept** coefficient is the starting point. If it reads 15.2, a student with 0% in PlusTwo is predicted to score 15.2% in Degree.
3. The **PlusTwo** coefficient is the slope. If it reads 0.85, then for every 1% rise in PlusTwo the predicted Degree mark rises by 0.85%.

The fitted line can be drawn on top of the scatter plot.

```
plt.scatter(df['PlusTwo'], df['Degree'], color='purple',
            ↪ alpha=0.5, label='Actual Students')

predicted_degree = model.predict(df['PlusTwo'])
plt.plot(df['PlusTwo'], predicted_degree, color='red',
         ↪ linewidth=3, label='Regression Trend Line')

plt.title("PlusTwo vs Degree with Best-Fit Trend Line")
plt.xlabel("PlusTwo Marks (%)")
plt.ylabel("Degree Marks (%)")
plt.legend()
plt.show()
```

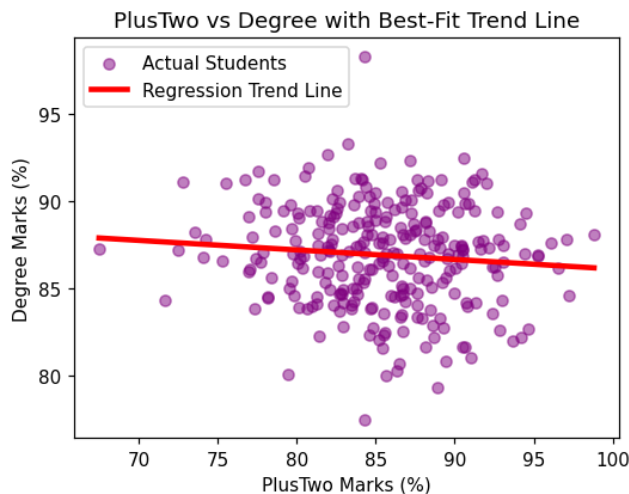


Figure 4.7: Scatter plot with the fitted regression trend line.

`model.predict(df['PlusTwo'])` uses the fitted equation to compute a predicted Degree mark for every student, and these predictions, joined together, form the straight red trend line.

4.10 Multiple Linear Regression

Multiple linear regression predicts one variable from **two or more** predictors at once. For example, we may predict a student's Degree mark from *both* their SSLC and PlusTwo marks together. The only change is that the formula lists more than one predictor, joined by a plus sign.

```
import statsmodels.formula.api as smf

# Predict Degree from BOTH SSLC and PlusTwo
model = smf.ols('Degree ~ SSLC + PlusTwo', data=df).fit()
print(model.summary())
```

The summary is read in the same way as before, with one extension: there is now a separate slope coefficient for **each** predictor. The SSLC coefficient shows how much the predicted Degree mark changes for each one-percent rise in SSLC *while PlusTwo is held fixed*, and the PlusTwo coefficient does the same for PlusTwo. The R-squared again reports the overall accuracy, now using both predictors together.

4.11 Logistic Regression

All the regressions so far predict a **number**. **Logistic regression** is used when the outcome is a **yes/no** category instead — for example, predicting whether a student is Female or Male. Instead of a number, it predicts a **probability** between 0 and 1.

Because Gender is already stored as 0 for Male and 1 for Female, it can be used directly as the yes/no outcome. Consider predicting Gender (Female or not) from the Degree mark.

```
import statsmodels.formula.api as smf

# Predict the yes/no outcome 'Gender' (1 = Female) from Degree
↪ marks
model = smf.logit('Gender ~ Degree', data=df).fit()
print(model.summary())
```

The function used is `smf.logit()` rather than `smf.ols()`, because the outcome is a category rather than a number. The output is read more simply here: look at the **sign** of the Degree coefficient. A **positive** coefficient means that as the Degree mark rises, the probability of the outcome (being Female) rises; a **negative** coefficient means the probability falls. The predicted probabilities trace out a smooth S-shaped curve rather than a straight line.

```
import numpy as np

# Predicted probability across the range of Degree marks
xs = np.linspace(df['Degree'].min(), df['Degree'].max(), 200)
predicted_prob = model.predict(pd.DataFrame({'Degree': xs}))

plt.scatter(df['Degree'], df['Gender'], alpha=0.3, color='gray',
            ↪ label='Actual (0=Male, 1=Female)')
plt.plot(xs, predicted_prob, color='red', linewidth=3,
        ↪ label='Predicted Probability (Female)')
plt.title("Logistic Regression: Predicting Gender from Degree")
plt.xlabel("Degree Marks (%)")
plt.ylabel("Probability of Female")
plt.legend()
plt.show()
```

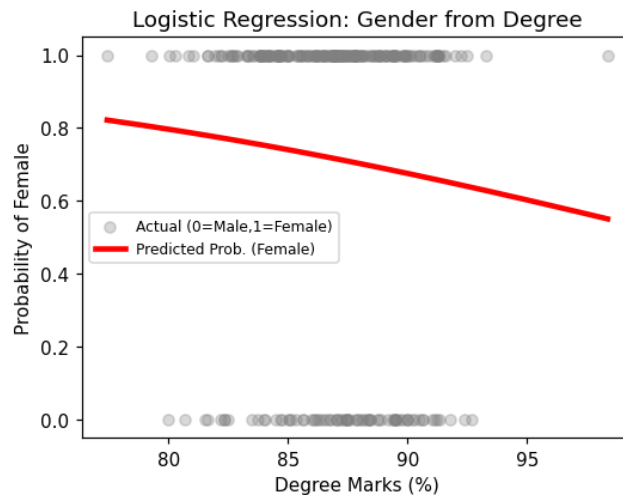


Figure 4.8: Logistic regression S-curve giving the predicted probability of being Female across Degree marks.

The grey dots at heights 0 and 1 are the actual students (Male at 0, Female at 1), and the red S-shaped curve gives the model's predicted probability of being Female at each Degree mark.

Practice Exercises

Exercise 4.1 — Check Normality of PlusTwo. Load the college data and run a Shapiro–Wilk test on the `PlusTwo` column. Print the p-value and state, using the 0.05 rule, whether the data is normal.

Exercise 4.2 — One-Sample Test. Using the result of Exercise 4.1, test whether the average `PlusTwo` mark is equal to 85%. Run the correct test (one-sample t-test if normal, Wilcoxon if not) and give your conclusion.

Exercise 4.3 — Two Groups. Map `Locality` to 'Urban' and 'Rural', separate the `SSLC` marks for the two groups, check their normality, and run the appropriate two-sample test (independent t-test or Mann–Whitney U). Print the p-value.

Exercise 4.4 — Three or More Groups. Separate the `Degree` scores by the four `Income` levels (0, 1, 2, 3) and run a Kruskal–Wallis test (`stats.kruskal`) to compare all four groups at once.

Exercise 4.5 — Correlation and Regression. Draw a scatter plot of `SSLC` (x-axis) against `PlusTwo` (y-axis), compute the correlation between them with `df['SSLC'].corr(df['PlusTwo'])`, then fit a simple linear regression `smf.ols('PlusTwo ~ SSLC', data=df).fit()` and note the R-squared value from the summary.

Glossary

Argument (actual argument) — the real value passed to a function when it is called.

Bin — one of the equal ranges into which a histogram groups numerical data.

Casting (type conversion) — converting a value from one data type to another, for example `int()`, `float()`, or `str()`.

Comment — text in a program that is ignored by Python; a single-line comment begins with `#`.

CSV — Comma-Separated Values, a plain-text file format for tabular data.

DataFrame — a two-dimensional, labelled, tabular data structure in Pandas.

Docstring — a multi-line string, enclosed in triple quotes, used for documentation.

Dynamic typing — the feature by which Python determines a variable's type automatically at runtime.

Escape sequence — a backslash combination inside a string, such as `\n` (new line) or `\t` (tab).

Exception — a runtime error that occurs while a syntactically correct program is running.

Expression — a combination of values, variables, and operators that evaluates to a result.

f-string — a formatted string literal, written with an `f` before the quote, in which variables inside `{}` are replaced by their values.

Function — a named block of reusable code that performs a specific task.

Index — the label attached to each value in a Pandas Series or each row of a DataFrame.

Indentation — the leading space that Python uses to define blocks of code.

Keyword (reserved word) — a word with a fixed meaning in Python, such as `if`, `for`, or `def`, that cannot be used as a variable name.

Loop — a structure (**for** or **while**) that repeats a block of code.

Module — a file of reusable Python code loaded with the **import** statement.

NaN — “Not a Number”, the value Pandas uses to mark missing data.

Parameter (formal parameter) — a placeholder variable listed in a function definition.

Series — a one-dimensional, labelled array in Pandas.

Split-apply-combine — the strategy behind `.groupby()`: splitting data by category, applying a calculation, and combining the results.

Syntax error — an error caused by breaking Python’s grammatical rules, which prevents the program from running.

Variable — a name that refers to a stored value in memory.

Alternative hypothesis (H1) — the claim that a real difference or effect exists.

ANOVA (Analysis of Variance) — a test comparing the means of three or more groups at once; one-way uses a single factor, two-way uses two factors.

Correlation coefficient (r) — a number between -1 and +1 measuring how strongly two numerical variables move together.

Hypothesis testing — using data as evidence to decide between a null and an alternative hypothesis.

Kruskal-Wallis test — the non-parametric alternative to one-way ANOVA.

Linear regression — a model predicting a numerical outcome from one predictor (simple) or several predictors (multiple).

Logistic regression — a model predicting a yes/no outcome as a probability between 0 and 1.

Mann-Whitney U test — the non-parametric alternative to the independent two-sample t-test.

Non-parametric test — a test used when data is not normally distributed; based on ranks rather than means.

Normal distribution — a symmetrical, bell-shaped spread of data in which most values cluster near the average.

Null hypothesis (H0) — the default assumption that no real difference or effect exists.

Parametric test — a test used when data is normally distributed, such as the t-test or ANOVA.

p-value — the probability of seeing the observed data by chance if the null hypothesis were true; compared against 0.05.

Paired test — a comparison of the same individuals measured twice (for example, before and after).

Regression — finding an equation that predicts one variable from one or more others.

Shapiro-Wilk test — a test that checks whether data is normally distributed.

Simulation — generating artificial data with numpy to imagine a scenario.

t-test — a parametric test comparing means: one-sample (against a fixed value), independent (two separate groups), or paired (same group twice).

Uniform distribution — a spread of data in which every value in a range is equally likely.

Wilcoxon signed-rank test — the non-parametric alternative to the one-sample and paired t-tests.

References

1. Embarak, O. (2018). *Data Analysis and Visualization Using Python*. Apress.
2. Gowrishankar, S., & Veena, A. (2018). *Introduction to Python Programming*. Chapman and Hall/CRC.
3. Guttag, J. V. (2016). *Introduction to Computation and Programming Using Python: With Application to Understanding Data*. MIT Press.
4. Haslwanter, T. (2016). *An Introduction to Statistics with Python: With Applications in the Life Sciences*. Springer International Publishing.
5. Lambert, K. A., & Osborne, M. (2015). *Fundamentals of Python*. Cengage Learning.
6. Lutz, M. (2013). *Learning Python: Powerful Object-Oriented Programming*. O'Reilly Media.
7. McKinney, W. (2012). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media.
8. Severance, C. (2016). *Python for Everybody: Exploring Data Using Python 3*. Charles Severance.
9. Tattar, P., Ojeda, T., Murphy, S. P., Bengfort, B., & Dasgupta, A. (2017). *Practical Data Science Cookbook*. Packt Publishing.
10. Unpingco, J. (2016). *Python for Probability, Statistics, and Machine Learning*. Springer International Publishing.
11. VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.